

Interval Scheduling with Eligible Machine Sets

Yuval Itzhaki

Claude

Abstract

In INTERVAL SCHEDULING ON ELIGIBLE MACHINES, a set of n jobs is to be processed non-preemptively on a set of m machines. Each job has a processing time, a deadline, a weight, and a set of eligible machines that can process it. The goal is to find a maximum weight subset of jobs that can each be processed on one of its eligible machines such that it completes exactly at its deadline. Hermelin, Itzhaki, Molter and Shabtay [3] showed that the problem is W[1]-hard when parameterized by the number m of machines, NP-hard even when the largest processing time $p_{\max}$ is constant, and fixed-parameter tractable for the combined parameter $m + p_{\max}$. All three results are proved in Lean. This note presents the definitions, constructions and statements as they are formalized, in the notation of [3], and links each of them to its Lean counterpart.

1 Introduction

In an interval scheduling problem each job can be processed only in a fixed time interval, and a schedule only decides which machine processes it. In INTERVAL SCHEDULING ON ELIGIBLE MACHINES every job j comes with a set M_j of eligible machines. Arkin and Silverberg [1] showed that the problem is strongly NP-hard and solvable in $O(n^{m+1})$ time, and whether it is fixed-parameter tractable for m was Open Problem 8 in the list of Mnich and van Bevern [4]. The paper [3] answers this question negatively (Theorem 1), shows that a constant $p_{\max}$ does not help either (Theorem 2), and gives an algorithm for the combination of both parameters (Theorem 3).

Running times are stated on the word RAM: a program is a fixed list of instructions operating on words of w bits, and its running time is the number of instructions it executes. A statement about a running time is a statement about one program, at every word length that admits the input.

2 Preliminaries

Definition 1 (INTERVAL SCHEDULING ON ELIGIBLE MACHINES). We have a set of n jobs $\{j_1, \dots, j_n\}$ and a set of m machines $\{i_1, \dots, i_m\}$, each of which can process one job at a time. Every job j has a processing time $p_j \geq 1$, a deadline $d_j \geq p_j$, a weight w_j , and a set of eligible machines $M_j \subseteq \{i_1, \dots, i_m\}$. Job j can be processed only in the interval $[d_j - p_j, d_j]$. We denote the largest processing time by $p_{\max} = \max_{1 \leq j \leq n} p_j$.

A *schedule* is a function $\sigma : \{j_1, \dots, j_n\} \rightarrow \{i_1, \dots, i_m, \perp\}$ with $\sigma(j) \in M_j \cup \{\perp\}$; job j is scheduled on machine $\sigma(j)$, or not scheduled if $\sigma(j) = \perp$. Two jobs j, j' are *in conflict* if $[d_j - p_j, d_j] \cap [d_{j'} - p_{j'}, d_{j'}] \neq \emptyset$. A schedule σ is *feasible* if there is no pair of conflicting jobs j, j' with $\sigma(j) = \sigma(j') \neq \perp$. Its weight is $w(\sigma) = \sum_{\sigma(j) \neq \perp} w_j$. The goal is to find a feasible schedule of maximum weight.

An instance is given to a word RAM as a word of numbers: n, m , the processing times, the deadlines, the weights, and the sets M_j as $n + 1$ offsets into one array that lists the eligible machines of each job in turn. In the decision version the word ends with the target weight W . For NP-hardness an instance is encoded as a binary word, with the sets M_j as an $n \times m$ matrix.

We consider three problems on these words: deciding whether there is a feasible schedule σ with $w(\sigma) \geq W$, parameterized by m ; the same problem parameterized by $m + p_{\max}$; and deciding whether there is a feasible schedule in which all jobs are scheduled, parameterized by $p_{\max}$.

Definition 2 (FPT and Parameterized Reductions). A parameterized problem is *fixed-parameter tractable* if there are a program, a constant c and a function g such that the program decides every input x with parameter k within $c \cdot g(k) \cdot (|x| + 1)$ instructions. A *parameterized reduction* from a problem P to a problem Q is a function on words that is computed by a program within such a bound, maps yes-instances to yes-instances and no-instances to no-instances, and maps an input with parameter k to one with parameter at most $h(k)$ for some function h .

The bound is linear in $|x|$, where $f(k) \cdot |x|^{O(1)}$ is usually allowed, so a problem that is fixed-parameter tractable in this sense is so in the usual sense, and a parameterized reduction in this sense is one in the usual sense.

Definition 3 (NP-Hardness on a Class of Instances). A scheduling problem is *NP-hard* if every language in **NP** has a polynomial-time many-one reduction to it. It is *NP-hard on a class $\mathcal{C}$* of instances if the reductions can be chosen to produce only instances in $\mathcal{C}$.

The reductions we construct are computed by word RAM programs in time polynomial in the number of bits of the input, which is polynomial time on a Turing machine.

3 W[1]-Hardness for Number m of Machines

Definition 4 (MULTICOLORED CLIQUE). We are given a k -partite graph $G = (V_0 \uplus V_1 \uplus \dots \uplus V_{k-1}, E)$ and are asked whether G contains a clique of size k . The k vertex parts are called *colors*, and the parameter is k .

MULTICOLORED CLIQUE parameterized by k is W[1]-hard [2].

Construction 1. Let $G = (V_0 \uplus V_1 \uplus \dots \uplus V_{k-1}, E)$ be a k -partite graph with n_G vertices. Let $<_\pi$ be the total ordering of $V := V_0 \uplus \dots \uplus V_{k-1}$ that orders the vertices by color and breaks ties by index, so that $u <_\pi v$ whenever $u \in V_\ell$, $v \in V_{\ell'}$ and $\ell < \ell'$. Let $\pi(v) \in \{1, \dots, n_G\}$ denote the ordinal position of v in $<_\pi$. Let $c_1 = n_G + 1$, $c_2 = (k - 1)n_G c_1 + n_G + 1$, and $c_3 = (kn_G + k^2 n_G)n_G c_2 + 1$.

We create $m = \binom{k}{2} + 1$ machines: one *edge selection machine* for each color combination ℓ, ℓ' with $\ell < \ell'$, and one *validation machine*. We create the following jobs.

- For each vertex $v \in V_\ell$ we create k *vertex jobs* $j_v^{(0)}, \dots, j_v^{(k-1)}$. The job $j_v^{(\ell)}$ has processing time $k + 2$, deadline $(k + 2)\pi(v) + 1$ and weight 1. For each color $\ell' \neq \ell$ the job $j_v^{(\ell')}$ has processing time 1, deadline $(k + 2)\pi(v) - \ell'$ and weight c_1 .
- For each edge $e = \{u, v\} \in E$ with $u \in V_\ell$, $v \in V_{\ell'}$ and $\ell < \ell'$ we create one *edge job* j_e with processing time $(k + 2)(\pi(v) - \pi(u)) - \ell + \ell' - 1$, deadline $(k + 2)\pi(v) - \ell - 1$ and weight $c_2(\pi(v) - \pi(u)) + c_3$.
- For each color combination $\ell < \ell'$ we create $|V_\ell| + |V_{\ell'}|$ *color combination jobs*. For $u \in V_\ell$ the job $j_u^{(\ell, \ell')}$ has processing time $(k + 2)\pi(u) - \ell' - 2$, deadline $(k + 2)\pi(u) - \ell' - 1$ and weight $c_2\pi(u)$. For $v \in V_{\ell'}$ the job $j_v^{(\ell, \ell')}$ has processing time $(k + 2)(n_G - \pi(v)) + \ell + 2$, deadline $(k + 2)n_G + 2$ and weight $c_2(n_G - \pi(v))$.

The validation machine is an eligible machine of all vertex jobs. The edge selection machine for color combination ℓ, ℓ' with $\ell < \ell'$ is a further eligible machine of the vertex jobs $j_u^{(\ell, \ell')}$ with

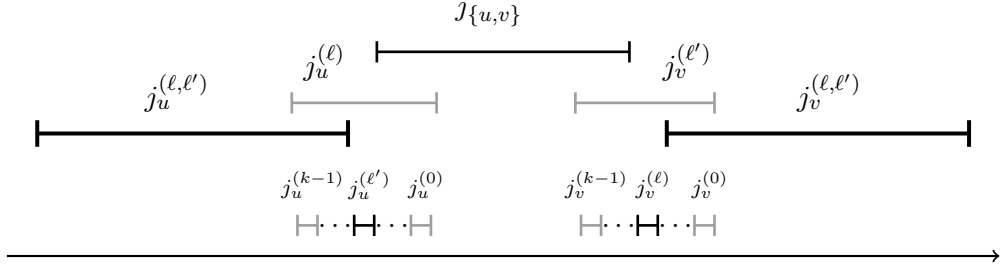


Figure 1: The edge selection machine for color combination ℓ, ℓ' with $\ell < \ell'$. Depicted are intervals of jobs relating to $u \in V_\ell$, $v \in V_{\ell'}$, and $e = \{u, v\} \in E$. Gray intervals correspond to jobs that are not eligible on the machine.

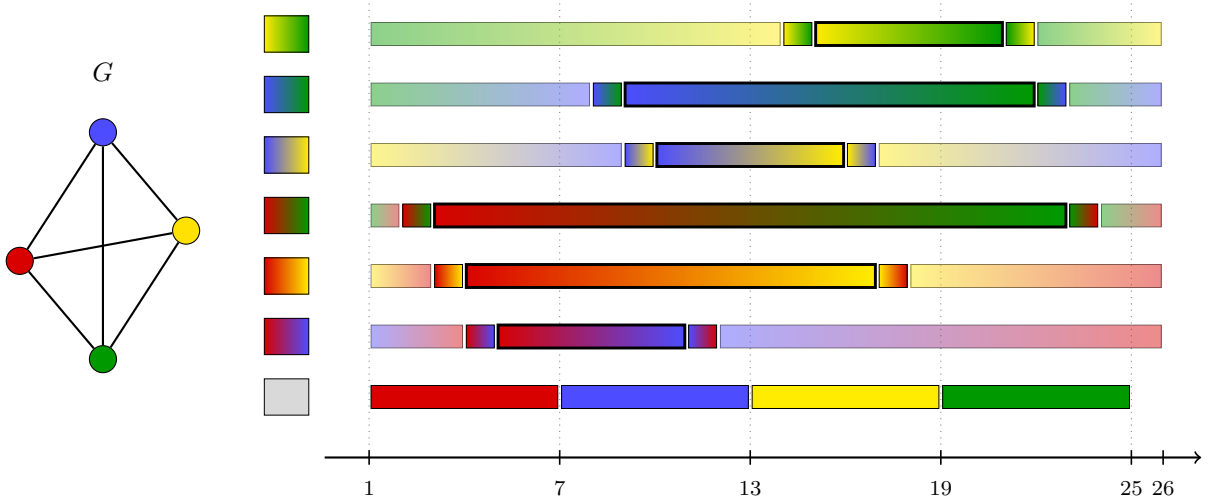


Figure 2: Construction 1 for a clique on $k = 4$ colors, drawn to scale, with the schedule of weight W . Each row is a machine: the validation machine (gray) processes the vertex job $j_v^{(\ell)}$ of every vertex, and the edge selection machine of each color combination processes a color combination job (pale), a unit vertex job, the edge job (bold), a unit vertex job and a color combination job.

$u \in V_\ell$ and $j_v^{(\ell)}$ with $v \in V_{\ell'}$, and it is the only eligible machine of the color combination jobs $j_u^{(\ell, \ell')}$, $j_v^{(\ell, \ell')}$ and of the edge jobs j_e with $e = \{u, v\}$, $u \in V_\ell$ and $v \in V_{\ell'}$. The target weight is

$$W = \binom{k}{2} c_3 + \binom{k}{2} n_G c_2 + (k-1) n_G c_1 + k.$$

Two details are done slightly differently than in [3]: the colors are numbered from 0, and the processing time of an edge job is one unit shorter. On the edge selection machine for ℓ, ℓ' the jobs $j_u^{(\ell, \ell')}$, $j_u^{(\ell')}$, j_e , $j_v^{(\ell)}$ and $j_v^{(\ell, \ell')}$ then have consecutive intervals, as illustrated in Figures 1 and 2.

Lemma 1 (Lemmas 1 and 2 of [3]). *Let I be the INTERVAL SCHEDULING ON ELIGIBLE MACHINES instance computed from G as specified by Construction 1. Then G contains a clique of size k if and only if there is a feasible schedule σ for I with $w(\sigma) \geq W$.*

Proof idea. Given a clique, schedule on the edge selection machine for ℓ, ℓ' the edge job of the clique edge between V_ℓ and $V_{\ell'}$ together with the four jobs around it, and schedule the vertex jobs $j_v^{(\ell)}$ of the clique vertices and all vertex jobs of the other vertices with weight c_1 on the validation machine. Conversely, a schedule of weight W must schedule exactly one edge job on

every edge selection machine, since c_3 exceeds the total weight of all other jobs. The weights c_2 force the two color combination jobs next to it to belong to its endpoints, and the weights c_1 force the $\binom{k}{2}$ selected edges to agree on one vertex per color. $\square$

Theorem 1. *MULTICOLORED CLIQUE parameterized by the number k of colors has a parameterized reduction to INTERVAL SCHEDULING ON ELIGIBLE MACHINES parameterized by the number m of machines. The reduction is Construction 1, and it produces $m = \binom{k}{2} + 1$ machines.*

Since MULTICOLORED CLIQUE is W[1]-hard, INTERVAL SCHEDULING ON ELIGIBLE MACHINES is W[1]-hard when parameterized by m . The class W[1] is not formalized; the formalized statement is the reduction.

4 NP-Hardness for Constant $p_{\max}$

Definition 5 ((3,4)-SAT). We are given a Boolean formula ϕ in conjunctive normal form where every clause has exactly three literals and every variable appears at most four times, and are asked whether ϕ has a satisfying assignment.

(3,4)-SAT is NP-hard [5]. For the reduction, ϕ is given as a word that lists, for the h th literal of each clause, its variable, whether it is negated, and which appearance of its variable it is.

Construction 2. Let ϕ be a formula as above. We construct an instance I of INTERVAL SCHEDULING ON ELIGIBLE MACHINES as follows.

- For each variable x we create two machines i_x^T, i_x^F and one job j_x with deadline and processing time 25 that is eligible on i_x^T and i_x^F .
- For each clause c we create three machines $i_c^{(1)}, i_c^{(2)}, i_c^{(3)}$ and nine jobs.
- Three of them are *literal jobs* $j_c^{(1)}, j_c^{(2)}, j_c^{(3)}$ with processing time one. Let the h th literal of c be the k th appearance of variable x . Then the deadline of $j_c^{(h)}$ is $2k + 8h - 8$. It is eligible on $i_c^{(2)}, i_c^{(3)}$, and on i_x^T if x appears non-negated and on i_x^F otherwise.
- For each literal job $j_c^{(h)}$ with deadline d we create two *wrapper jobs*: $j_{c,\alpha}^{(h)}$ with deadline and processing time $d - 1$, and $j_{c,\omega}^{(h)}$ with deadline 25 and processing time $25 - d$. All wrapper jobs are eligible on $i_c^{(1)}, i_c^{(2)}, i_c^{(3)}$.

All jobs have weight one. See Figure 3 for an illustration.

Lemma 2 (Lemmas 3 and 4 of [3]). *Let I be the INTERVAL SCHEDULING ON ELIGIBLE MACHINES instance computed from ϕ as specified by Construction 2. Then ϕ is satisfiable if and only if there is a feasible schedule for I such that all jobs are scheduled.*

Proof idea. Job j_x blocks one of i_x^T, i_x^F for the whole time; we set x to true if and only if j_x is scheduled on i_x^F . In a schedule of all jobs the two wrapper jobs of each literal job are on the same machine, so the machines $i_c^{(2)}, i_c^{(3)}$ have room for two of the three literal jobs of c , and the third one is scheduled on a machine of its variable, which makes its literal true. Distinct appearances of a variable have non-overlapping literal jobs, so a satisfying assignment yields a schedule of all jobs. $\square$

Theorem 2. *It is NP-hard to decide whether all jobs of a INTERVAL SCHEDULING ON ELIGIBLE MACHINES instance can be scheduled, even when $p_{\max} \leq 25$ and all jobs have unit weight.*

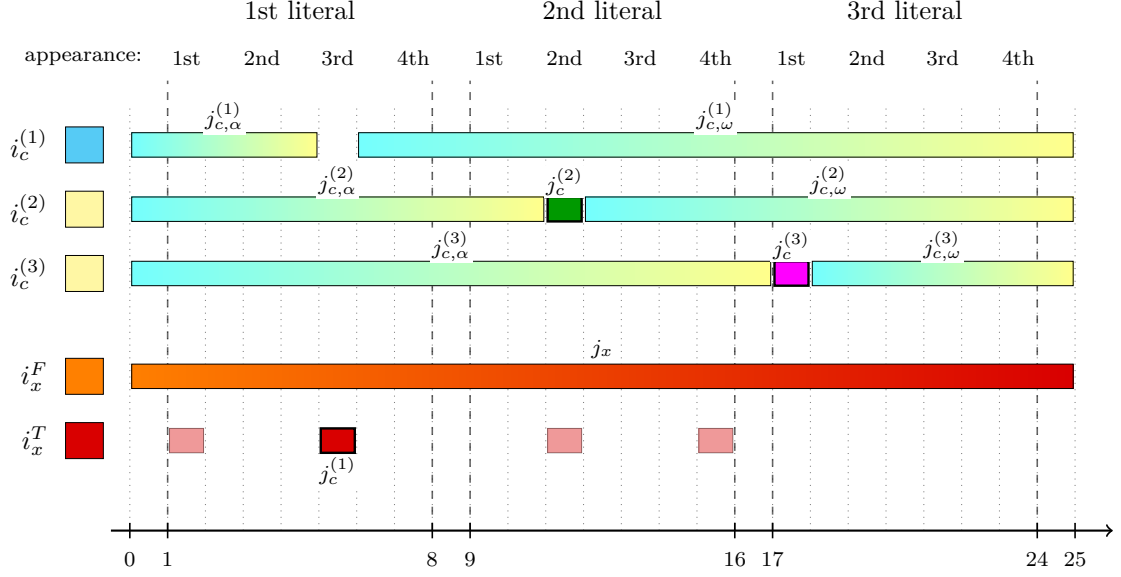


Figure 3: Construction 2 applied to the clause $c = (x \vee y \vee \neg z)$, which contains the third appearance of x , the second appearance of y , and the first appearance of z . The dashed lines delimit the slots of the literal jobs. Shown is a schedule for an assignment that sets x to true: j_x is on i_x^F , the literal job $j_c^{(1)}$ is on i_x^T together with the literal jobs of other appearances of x (pale), and its two wrapper jobs are on $i_c^{(1)}$.

5 FPT for the Combination of m and $p_{\max}$

Construction 3. Let I be an instance of INTERVAL SCHEDULING ON ELIGIBLE MACHINES. For every interval $[r, d)$ and every machine i let $S_{[r,d)}^{(i)}$ denote the set of the m highest-weighted jobs that have interval $[r, d)$ and are eligible on machine i , where ties are broken by the index of the job. All jobs that are not contained in any set $S_{[r,d)}^{(i)}$ are discarded.

Lemma 3 (Lemma 5 of [3]). *Let I' be obtained from I by applying Construction 3. Then the maximum weight of a feasible schedule is the same for I and I' .*

Observation 1 (Observation 5 of [3]). *Let I' be obtained from I by applying Construction 3. Then for every time step t there are at most $m^2 p_{\max}^2$ jobs in I' whose interval contains t .*

For a time step t let J_t be the set of jobs whose interval contains t . The dynamic programming table $T_t : (J_t \cup \{\perp\})^m \rightarrow \mathbb{N}$ is defined as follows: the entry $T_t[\ell_1, \dots, \ell_m]$ is the maximum weight of the scheduled jobs that start at or before t , over all feasible schedules such that for each $i \in \{1, \dots, m\}$, at time t machine i processes job ℓ_i if $\ell_i \neq \perp$ and is idle if $\ell_i = \perp$. An entry is *valid* if $i \in M_{\ell_i}$ for all i with $\ell_i \neq \perp$ and no job appears twice. For a valid entry, $T_0[\ell_1, \dots, \ell_m]$ is the total weight of the jobs $\ell_1, \dots, \ell_m$, and

$$T_{t+1}[\ell'_1, \dots, \ell'_m] = \max_{[\ell_1, \dots, \ell_m] \rightarrow [\ell'_1, \dots, \ell'_m]} T_t[\ell_1, \dots, \ell_m] + \sum_{\substack{j \in \{\ell'_1, \dots, \ell'_m\} \\ d_j - p_j = t+1}} w_j,$$

where the maximum is over the valid entries of T_t such that every job ℓ_i that is still processed at time $t+1$ satisfies $\ell'_i = \ell_i$, and every job ℓ'_i that starts at or before t satisfies $\ell_i = \ell'_i$. The table T_t has at most $(|J_t| + 1)^m$ entries.

Lemma 4 (Lemma 6 of [3]). *The recursion above correctly computes $T_t[\ell_1, \dots, \ell_m]$ for every time step t and every valid entry. In particular, for $D = \max_j d_j$ the entry $T_D[\perp, \dots, \perp]$ is the maximum weight of a feasible schedule.*

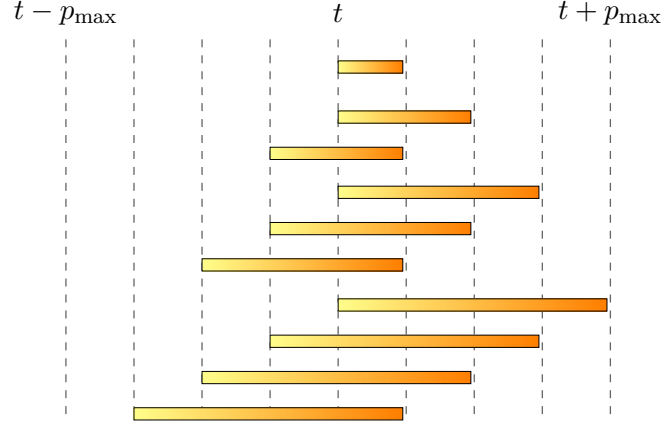


Figure 4: There are at most $p_{\max}^2$ distinct intervals of length at most $p_{\max}$ that contain time step t .

Theorem 3. *There are a word RAM program and a constant c such that for every instance x of INTERVAL SCHEDULING ON ELIGIBLE MACHINES with target weight W , at every word length w satisfying the input admissibility condition and $c(mp_{\max} + 1)^{2m} \leq 2^w$, the program decides whether there is a feasible schedule σ with $w(\sigma) \geq W$ within*

$$c \cdot (m \cdot p_{\max} + 1)^{2m} \cdot (m + 1) \cdot (|x| + 1)$$

instructions. A separate program decides all admissible inputs in fixed-parameter time for $m + p_{\max}$, using exhaustive search when the configuration table cannot be addressed.

The explicit bound corresponds to $O((m \cdot p_{\max})^{2m} \cdot mn)$ in [3], with input length $|x|$ in place of the number of jobs and an address-space condition: by Observation 1 we have $|J_t| \leq m^2 p_{\max}^2$ after applying Construction 3, so every table has at most $(m \cdot p_{\max} + 1)^{2m}$ entries.

References

- [1] E. M. Arkin and E. B. Silverberg. Scheduling Jobs with Fixed Start and End Times. *Discrete Applied Mathematics* 18(1):1–8, 1987. DOI: [10.1016/0166-218X\(87\)90037-0](https://doi.org/10.1016/0166-218X(87)90037-0)
- [2] M. R. Fellows, D. Hermelin, F. Rosamond and S. Vialette. On the Parameterized Complexity of Multiple-Interval Graph Problems. *Theoretical Computer Science* 410(1):53–61, 2009. DOI: [10.1016/j.tcs.2008.09.065](https://doi.org/10.1016/j.tcs.2008.09.065)
- [3] D. Hermelin, Y. Itzhaki, H. Molter and D. Shabtay. On the Parameterized Complexity of Interval Scheduling with Eligible Machine Sets. *Journal of Computer and System Sciences* 144:103533, 2024. DOI: [10.1016/j.jcss.2024.103533](https://doi.org/10.1016/j.jcss.2024.103533)
- [4] M. Mnich and R. van Bevern. Parameterized Complexity of Machine Scheduling: 15 Open Problems. *Computers & Operations Research* 100:254–261, 2018. DOI: [10.1016/j.cor.2018.07.020](https://doi.org/10.1016/j.cor.2018.07.020)
- [5] C. A. Tovey. A Simplified NP-Complete Satisfiability Problem. *Discrete Applied Mathematics* 8(1):85–89, 1984. DOI: [10.1016/0166-218X\(84\)90081-7](https://doi.org/10.1016/0166-218X(84)90081-7)